Memory Allocation(Theoretical Vs Reality)

We have seen:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

- ***→ data(an integer) is typically 4 bytes.***
- ***→ npx(a pointer to another Node) is typically 8 bytes.***

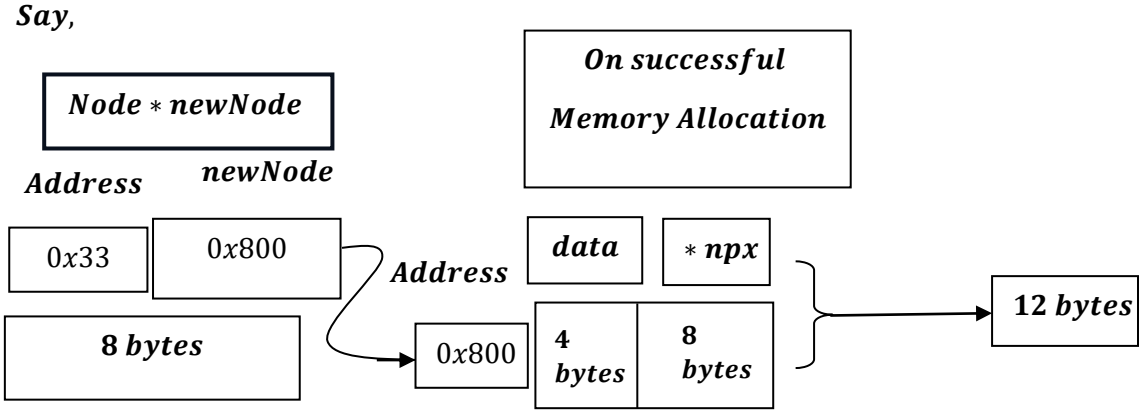
→ Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes , npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real-World Allocation

In a real programming environment $\begin{pmatrix} \textbf{like the C++ environments} \\ \textbf{you typically work in} \end{pmatrix}$

, ***the allocation might not be exactly 12 bytes because of***

how CPUs access memory:

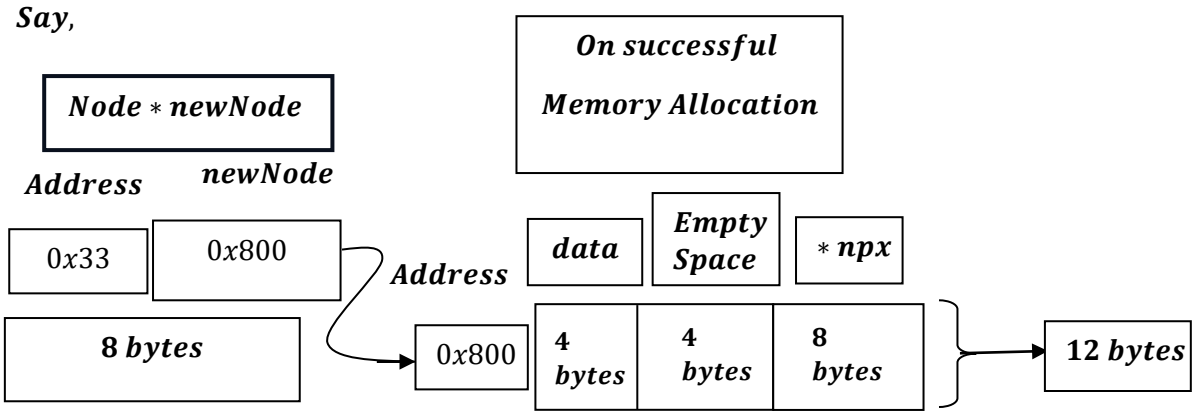
- ***Memory Alignment***: *Most* $64-bit$ *compilers align data to* $8-$ *byte boundaries to improve performance*.

- ***Padding***: *To align the* $8-$ *byte* npx *pointer, the compiler will often add* 4 *bytes of "padding" after the* $4-$ *byte* $data$ *integer*.

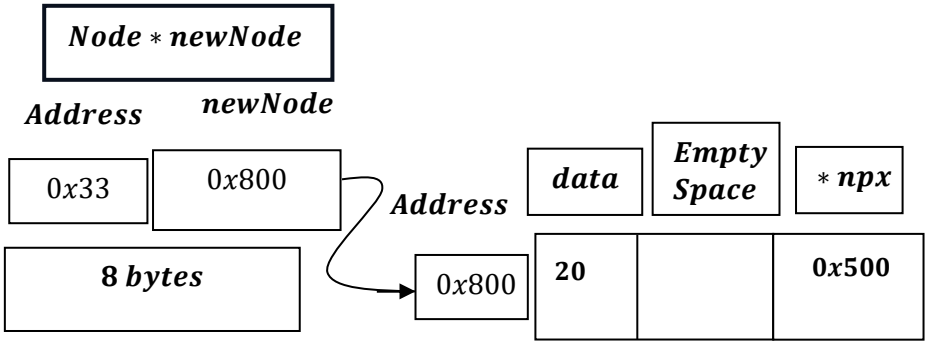
- ***Actual Size***: *Consequently,* $sizeof(Node)$ *would likely return* 16 *bytes on your system, rather than the* 12 *bytes shown in the document*.

Offset	*Size*	*Content*	*Description*
0	**4 *bytes***	***data***	***The integer value* (*e.g.*, 2).**
4	**4 *bytes***	***Padding***	***Empty space added by the compiler*.**
8	**8 *bytes***	***npx***	***The XORed address* (*e.g.*, 0*x*500).**

***It will look like*:**



***Like*:**



In a 64 − bit environment, the compiler aligns memory to the size of the largest primitive member in a structure − in this case, the 8 − byte npx pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty***space (padding) to ensure the pointer starts at an address that is a multiple of 8.***

Why it is Memory Efficient?

1. *The Real Specialty*: *Memory Efficiency*

In a standard Doubly Linked List, each node requires ***two*** *pointers: next and prev.*

- ***Standard DLL Node***: $[prev \mid data \mid next] \rightarrow 2\ pointers + 1\ data\ field.$
- ***XOR Node***: $[npx \mid data] \rightarrow 1\ pointer\ field + 1\ data\ field.$

By using the XOR bitwise operation $(A \oplus B)$

, we compress two addresses into a single memory space (npx).

Feature	*Standard Doubly Linked List*	*XOR Linked List*
Pointers per Node	2 $(next, prev)$	1 (npx)
Memory Usage	*Higher*	*Lower* $(saves\ \sim 50\%\ pointer\ space)$
Traversal	*Bidirectional* (*Simple*)	*Bidirectional* (*Requires bitwise math*)
Readability	*High*	*Low* (*Complex logic*)

2. *Does it reduce LOC? (Spoiler: Usually No)*

Actually, the XOR Linked List often requires ***more lines of code***

than a standard list. This is because:

- ***Pointer Arithmetic***: *We cannot simply say* $temp = temp \rightarrow next$.
  We must perform an XOR calculation using the current node
  and the previous node's address every time you move.

- ***Type Casting***: *C++ pointers cannot be XORed directly.*
  We must cast them to $uintptr\_t$, *perform the math,*
  and cast them back to a pointer.

- ***Helper Functions***: *We usually need to write a specific* $XOR()$
  helper function to keep the code readable.

Comparison Example (Traversal Logic):

Standard DLL:

```
        current = current ->next;
```

XOR Linked List:

```
    Node *next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
```

3. *Why use it if it's harder to code?*

The XOR Linked List is a niche*optimization used in environments*

where ***every byte of RAM counts****, such as:*

1. ***Embedded Systems***: *Devices with extremely limited memory (kilobytes of RAM).*

2. ***Large − scale Databases***: *When storing billions of nodes,*

   saving $4 - 8$ *bytes per node adds up to gigabytes of saved memory.*

3. ***Old − school Game Development***: *Where hardware limitations*
   forced developers to be extremely clever with bit manipulation.

The XOR Linked List is a Space – Optimization technique, not a code – shortening technique. It makes your program use less RAM at the cost of making the code more difficult to read and maintain.
